a browser. Using this module, you will be able to send arbitrary data to the browser at specific points in time.
The default Rails server (WEBrick) is a buffering web server and does not support streaming. In order to use this feature, you'll need to use a non buffering server like Puma,Rainbows or Passenger.

12.3.1 Incorporating Live Streaming

Including `ActionController::Live` inside of your controller class will provide all actions inside of the controller the ability to stream data. You can mix in the module like so:

```
class MyController < ActionController::Base
  include ActionController::Live

  def stream
    response.headers['Content-Type'] = 'text/event-stream'
    100.times {
      response.stream.write "hello world\n"
      sleep 1
    }
  ensure
    response.stream.close
  end
end
```

The above code will keep a persistent connection with the browser and send 100 messages of `"hello world\n"`, each one second apart.
There are a couple of things to notice in the above example. We need to make sure to close the response stream. Forgetting to close the stream will leave the socket open forever. We also have to set the content type to `text/event-stream` before we write to the response stream. This is because headers cannot be written after the response has been committed (when `response.committed`returns a truthy value), which occurs when you `write` or `commit` the response stream.

12.3.2 Example Usage

Let's suppose that you were making a Karaoke machine and a user wants to get the lyrics for a particular song. Each `Song` has a particular number of lines and each line takes time `num_beats` to finish singing.
If we wanted to return the lyrics in Karaoke fashion (only sending the line when the singer has finished the previous line), then we could use `ActionController::Live` as follows:

```
class LyricsController < ActionController::Base
  include ActionController::Live

  def show
    response.headers['Content-Type'] = 'text/event-stream'
    song = Song.find(params[:id])

    song.each do |line|
      response.stream.write line.lyrics
      sleep line.num_beats
    end
  ensure
    response.stream.close
  end
end
```

The above code sends the next line only after the singer has completed the previous line.

12.3.3 Streaming Considerations

Streaming arbitrary data is an extremely powerful tool. As shown in the previous examples, you can choose when and what to send across a response stream. However, you should also note the following things: